universidade de aveiro
estga
escola superior de tecnologia
e gestão de águeda

Comunicação e Sincronização entre Processos

Sistemas Operativos

Licenciatura em Engenharia Informática Aplicada

Juan Carlos Gonsalves Da Silva

Águeda | 25 de Maio de 2025

universidade de aveiro
estga
escola superior de tecnologia
e gestão de águeda

Comunicação e Sincronização entre Processos

Sistemas Operativos

Licenciatura em Engenharia Informática Aplicada

Juan Carlos Gonsalves Da Silva

Águeda | 25 de Maio de 2025

Índice

1. Introdução....4
2. Objetivos....5
3. Arquitetura do Sistema....6
Processos....6
4. Lógica de Triagem e Atendimento....9
5. Interface Gráfica....10
6. Hospital.log....11
7. Eventos Especiais....12
Surtos....12
Desistências....12
8. Dificuldades....13
9. Conclusões....13

1. Introdução

O presente relatório descreve o desenvolvimento de uma aplicação de simulação de um serviço de urgências hospitalares, implementada no âmbito da unidade curricular de Sistemas Operativos. O principal objetivo do projeto foi aplicar conceitos de **processamento concorrente**, **memória partilhada** e **sincronização de processos** em Python, simulando o comportamento de pacientes e médicos num ambiente hospitalar.

A aplicação é composta por um processo servidor, que realiza a triagem, gestão da fila de espera e distribuição de pacientes pelos médicos, e por múltiplos processos clientes, que representam os pacientes que chegam ao hospital. Cada paciente possui um nível de urgência aleatório, e o sistema assegura que os casos mais graves são tratados com prioridade. Todos os eventos são registados num ficheiro de log.

Este relatório detalha a arquitetura da solução, as decisões técnicas tomadas durante o desenvolvimento, os principais desafios enfrentados e propostas de melhorias para versões futuras.

2. Objetivos

- Implementar um sistema baseado em processos concorrentes;
- Usar memória partilhada para comunicação de estados entre processos;
- Gerir uma fila de espera com prioridades baseadas em urgência;
- Registar eventos do sistema em tempo real;
- Fornecer uma interface gráfica informativa e visual.

3. Arquitetura do Sistema

O sistema é constituído por três módulos principais:

- **main.py**: ponto de entrada da aplicação. Inicia os processos de geração e atendimento de pacientes e executa a interface gráfica.
- **hospital_core.py**: contém a lógica principal do sistema, incluindo a criação de pacientes, a gestão de urgências e o atendimento pelos médicos.
- **interface.py**: define a interface gráfica, mostrando listas de pacientes e o log de eventos.

Processos

- **Processo Gerador de Pacientes:** cria pacientes aleatoriamente, com diferentes níveis de urgência, respeitando o limite de 20 pacientes em espera.

```python
# Processo responsável por gerar pacientes aleatórios
def gerar_pacientes(fila_pacientes, fila_visual, estado_pacientes):
    id_paciente = 1
    while True:
        # Conta quantos pacientes estão atualmente esperando
        total_esperando = sum(1 for estado in estado_pacientes.values() if estado == 'esperando')
        if total_esperando >= 20:
            time.sleep(1)
            continue

        if random.random() < 0.05:  # 5% de chance de "surto" onde são gerados mais pacientes
            inicio_surto = time.time()
            fila_visual.put(("SURTO", hora_atual()))
            while time.time() - inicio_surto < 10:
                for _ in range(random.randint(3, 5)):
                    if sum(1 for estado in estado_pacientes.values() if estado == 'esperando') >= 20:
                        break # se já ha 20 pacientes, não cria mais
                    p = Process(target=paciente_processo, args=(id_paciente, fila_pacientes, fila_visual, estado_pacientes))
                    p.start()
                    id_paciente += 1
                time.sleep(random.uniform(1, 4))
        else:
            for _ in range(random.randint(1, 3)):
                if sum(1 for estado in estado_pacientes.values() if estado == 'esperando') >= 20:
                    break # se já ha 20 pacientes, não cria mais
                p = Process(target=paciente_processo, args=(id_paciente, fila_pacientes, fila_visual, estado_pacientes))
                p.start()
                id_paciente += 1
            time.sleep(random.uniform(1, 4))
```

- **Processo de Atendimento:** gere a atribuição de médicos a pacientes com base na prioridade.

```python
# Função que simula o atendimento de um paciente
def atendimento(paciente, fila_visual, estado_pacientes):
    prioridade, id, nome, urg, tempo_inicial, tempo_max_espera = paciente
    estado_pacientes[id] = 'atendendo'
    fila_visual.put(('atendiendo', id, nome, urg, hora_atual()))
    time.sleep(random.randint(5, 10))  # Tempo de atendimento
    estado_pacientes[id] = 'atendido'
    fila_visual.put(('atendido', id, nome, urg, hora_atual()))
```

```python
# Processo principal responsável por gerenciar os atendimentos
def atender_pacientes(fila_pacientes, fila_visual, estado_pacientes):
    processos = []  # Armazena processos de atendimento ativos

    while True:
        # Remove processos que já terminaram
        processos = [p for p in processos if p.is_alive()]
        medicos_livres = MAX_MEDICOS - len(processos)

        if medicos_livres > 0:
            pacientes_disponiveis = []
            while not fila_pacientes.empty():
                paciente = fila_pacientes.get()
                tempo_inicial = paciente[4]
                tempo_max_espera = paciente[5]
                id = paciente[1]
                if time.time() - tempo_inicial < tempo_max_espera and estado_pacientes.get(id) == 'esperando':
                    pacientes_disponiveis.append(paciente)

            if pacientes_disponiveis:
                pacientes_disponiveis.sort()  # Menor prioridade (mais urgente) vem primeiro
                para_atender = pacientes_disponiveis[:medicos_livres]
                restante = pacientes_disponiveis[medicos_livres:]

                for p in restante:
                    if estado_pacientes.get(p[1]) == 'esperando':
                        fila_pacientes.put(p)

                for paciente in para_atender:
                    p = Process(target=atendimento, args=(paciente, fila_visual, estado_pacientes))
                    p.start()
                    processos.append(p)

        time.sleep(1)
```

- **Subprocessos de Pacientes:** cada paciente é representado por um processo que espera atendimento ou desiste após um tempo.

```python
# Processo individual que representa a vida de um paciente
def paciente_processo(id, fila_pacientes, fila_visual, estado_pacientes):
    nome = gerar_nome_completo()
    urg = gerar_urgencia()
    prioridade = urgencias[urg]
    tempo_max_espera = random.randint(15, 20)  # tempo que o paciente pode esperar antes de desistir, em segundos
    tempo_inicial = time.time()

    estado_pacientes[id] = 'esperando'
    fila_pacientes.put((prioridade, id, nome, urg, tempo_inicial, tempo_max_espera))
    fila_visual.put(('esperando', id, nome, urg, hora_atual()))

    # Espera até ser atendido ou desiste
    while time.time() - tempo_inicial < tempo_max_espera:
        if estado_pacientes.get(id) != 'esperando':
            return
        time.sleep(1)

    # Se ainda não foi atendido, desiste
    if estado_pacientes.get(id) == 'esperando':
        estado_pacientes[id] = 'desistiu'
        fila_visual.put(('desistiu', id, nome, urg, hora_atual()))
```

4. Lógica de Triagem e Atendimento

Cada paciente tem:

- Um **nome aleatório** gerado a partir de listas de nomes e apelidos.

```
# Listas de nomes e apelidos usados para gerar pacientes aleatórios
nomes = ['Ana<3', 'Carlos', 'Mariana', 'José', 'Paula', 'Miguel', 'Laura', 'Tiago', 'Sofia', 'Rui',
         'Pedro', 'Rafael', 'Thomaz', 'Felipe', 'Manuel', 'Mario', 'Rosa', 'Beatriz', 'Diana']
apelidos = ['Silva', 'Santos', 'Ferreira', 'Pereira', 'Costa', 'Oliveira', 'Martins', 'Rocha',
            'Almeida', 'Ramos', 'Vasco', 'Araujo', 'Liendo', 'Cambin', 'Fitorra']
```

- Um **nível de urgência** atribuído aleatoriamente: Vermelha, Amarela ou Verde.

```
# Dicionário que associa cada cor de urgência a uma prioridade (quanto menor, mais urgente)
urgencias = {'Verde': 2, 'Amarela': 1, 'Vermelha': 0}
```

As prioridades são geridas numericamente:

  - Vermelha: 0 (mais urgente);
  - Amarela: 1;
  - Verde: 2 (menos urgente).
- Um **tempo máximo de espera**, após o qual desiste se não for atendido.

```
# Processo individual que representa a vida de um paciente
def paciente_processo(id, fila_pacientes, fila_visual, estado_pacientes):
    nome = gerar_nome_completo()
    urg = gerar_urgencia()
    prioridade = urgencias[urg]
    tempo_max_espera = random.randint(15, 20)  # tempo que o paciente pode esperar antes de desistir, em segundos
    tempo_inicial = time.time()
```

O processo de atendimento atribui os médicos disponíveis (máximo de 5) aos pacientes com maior prioridade. Cada atendimento dura entre 5 a 10 segundos.

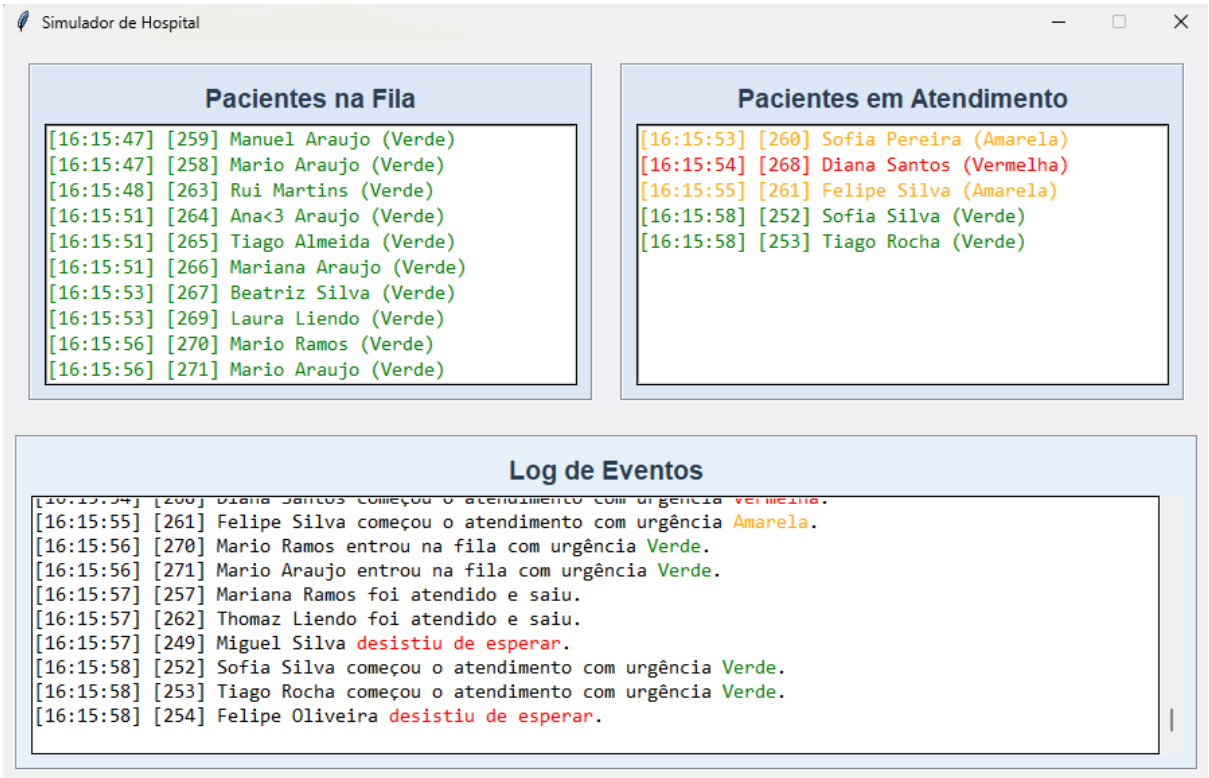
```
# Número máximo de médicos disponíveis ao mesmo tempo
MAX_MEDICOS = 5
```

```
# Função que simula o atendimento de um paciente
def atendimento(paciente, fila_visual, estado_pacientes):
    prioridade, id, nome, urg, tempo_inicial, tempo_max_espera = paciente
    estado_pacientes[id] = 'atendendo'
    fila_visual.put(('atendiendo', id, nome, urg, hora_atual()))
    time.sleep(random.randint(5, 10))  # Tempo de atendimento
    estado_pacientes[id] = 'atendido'
    fila_visual.put(('atendido', id, nome, urg, hora_atual()))
```

5. Interface Gráfica

A interface criada no ficheiro interface.py está dividida em:

- **Pacientes na Fila**: apresentados com cores diferentes por urgência.
- **Pacientes em Atendimento**: também com destaque visual.
- **Log de Eventos**: apresenta cada evento do hospital.



A interface é atualizada a cada segundo com base na fila visual. O ficheiro hospital.log armazena um registo persistente de todos os eventos.

6. Hospital.log

O log inclui:

- Chegada de pacientes e entrada na fila.
- Início e fim do atendimento.
- Desistências por tempo de espera.
- Ocorrência de surtos.

hospital.log

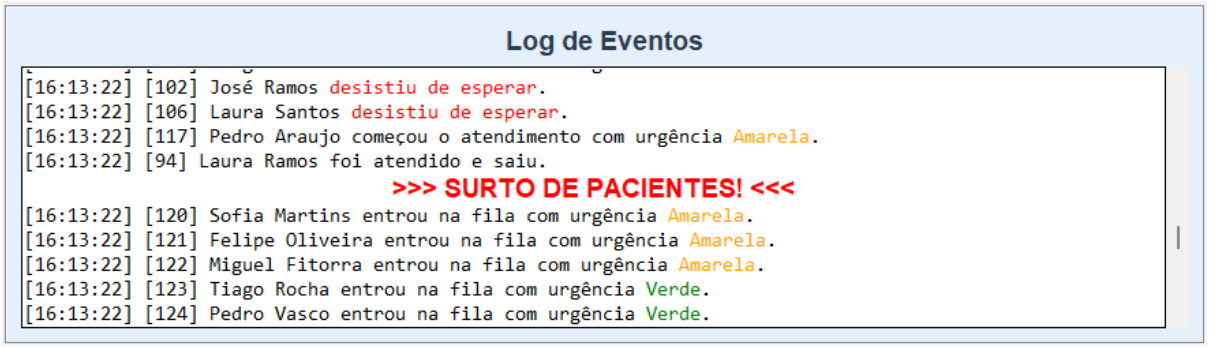
```
[16:10:56] [1] Beatriz Vasco entrou na fila com urgência Verde.
[16:10:57] [1] Beatriz Vasco começou o atendimento com urgência Verde.
[16:10:59] [2] Mariana Martins entrou na fila com urgência Verde.
[16:11:00] [2] Mariana Martins começou o atendimento com urgência Verde.
[16:11:01] [3] Carlos Pereira entrou na fila com urgência Amarela.
[16:11:03] [3] Carlos Pereira começou o atendimento com urgência Amarela.
[16:11:03] [4] Rui Oliveira entrou na fila com urgência Amarela.
[16:11:04] [4] Rui Oliveira começou o atendimento com urgência Amarela.
[16:11:04] [1] Beatriz Vasco foi atendido e saiu.
[16:11:05] [5] Pedro Fitorra entrou na fila com urgência Verde.
[16:11:06] [5] Pedro Fitorra começou o atendimento com urgência Verde.
[16:11:07] [2] Mariana Martins foi atendido e saiu.
[16:11:08] [6] Carlos Silva entrou na fila com urgência Amarela.
[16:11:09] [6] Carlos Silva começou o atendimento com urgência Amarela.
[16:11:09] [7] Rafael Silva entrou na fila com urgência Verde.
[16:11:09] [8] Rui Liendo entrou na fila com urgência Verde.
[16:11:10] [7] Rafael Silva começou o atendimento com urgência Verde.
[16:11:11] [3] Carlos Pereira foi atendido e saiu.
[16:11:11] [4] Rui Oliveira foi atendido e saiu.
[16:11:11] [9] Mariana Araujo entrou na fila com urgência Amarela.
[16:11:11] [10] Manuel Oliveira entrou na fila com urgência Amarela.
[16:11:12] [9] Mariana Araujo começou o atendimento com urgência Amarela.
[16:11:12] [10] Manuel Oliveira começou o atendimento com urgência Amarela.
[16:11:12] [11] Thomaz Santos entrou na fila com urgência Verde.
[16:11:14] [12] Sofia Rocha entrou na fila com urgência Amarela.
[16:11:14] [13] Diana Pereira entrou na fila com urgência Vermelha.
[16:11:14] [14] Tiago Rocha entrou na fila com urgência Verde.
[16:11:16] [5] Pedro Fitorra foi atendido e saiu.
[16:11:16] [6] Carlos Silva foi atendido e saiu.
[16:11:17] [7] Rafael Silva foi atendido e saiu.
[16:11:17] [13] Diana Pereira começou o atendimento com urgência Vermelha.
[16:11:17] [12] Sofia Rocha começou o atendimento com urgência Amarela.
[16:11:17] [9] Mariana Araujo foi atendido e saiu.
```

7. Eventos Especiais

Surtos

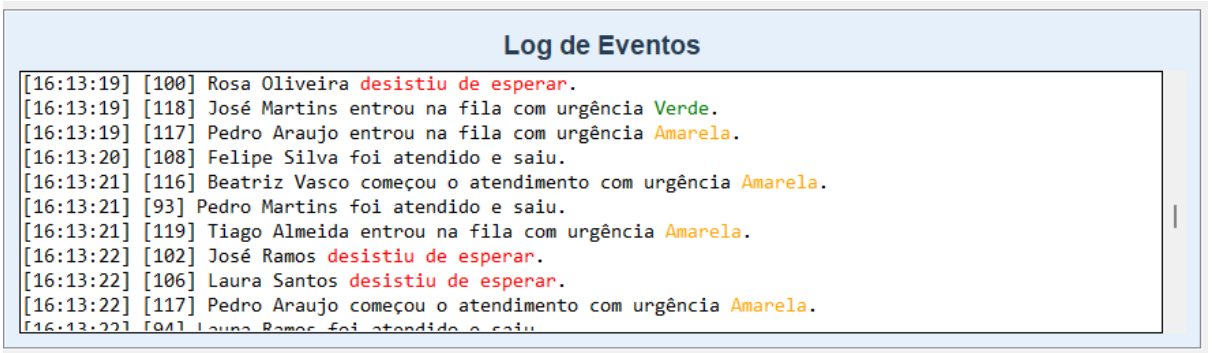
A aplicação simula "**surtos**" com 5% de probabilidade, em que vários pacientes chegam em pouco tempo. Durante 10 segundos, podem ser gerados até 5 pacientes de uma vez. Estes eventos são destacados visualmente no Log de Eventos.

```python
if random.random() < 0.05:  # 5% de chance de "surto" onde são gerados mais pacientes
    inicio_surto = time.time()
    fila_visual.put(("SURTO", hora_atual()))
    while time.time() - inicio_surto < 10:
        for _ in range(random.randint(3, 5)):
            if sum(1 for estado in estado_pacientes.values() if estado == 'esperando') >= 20:
                break # se já ha 20 pacientes, não cria mais
            p = Process(target=paciente_processo, args=(id_paciente, fila_pacientes, fila_visual, estado_pacientes))
            p.start()
            id_paciente += 1
        time.sleep(random.uniform(1, 4))
```



Desistências

Se um paciente esperar demasiado tempo sem ser atendido (tempo aleatório entre 15 e 20 segundos), desiste e é removido da fila. Esta ação também é registada visualmente e no log.




```python
# Se ainda não foi atendido, desiste
if estado_pacientes.get(id) == 'esperando':
    estado_pacientes[id] = 'desistiu'
    fila_visual.put(('desistiu', id, nome, urg, hora_atual()))
```

8. Dificuldades

Durante o desenvolvimento deste projeto, enfrentei algumas dificuldades técnicas que me fizeram refletir bastante sobre o funcionamento interno do sistema. A parte mais desafiadora, sem dúvidas, foi **garantir que os pacientes entrassem corretamente em atendimento**. Como o sistema usa múltiplos processos simultâneos, houve momentos em que pacientes ficavam presos na fila ou simplesmente não eram atendidos mesmo havendo médicos disponíveis. O problema estava na lógica que verifica os tempos de espera e o estado de cada paciente, e fazer com que essa verificação fosse precisa e justa exigiu uma reestruturação da fila e uma manipulação mais cuidadosa do dicionário de estados compartilhados.

Além disso, tive alguma dificuldade com os **detalhes da interface gráfica**, fazer com que todos os processos se refletissem em tempo real na interface visual foi complicado. Sincronizar corretamente os dados entre os processos e garantir que os eventos, como "esperando", "atendendo", "desistiu", etc., fossem exibidos sem atrasos ou erros, exigiu diversos testes e ajustes.

No geral, o projeto me ensinou muito sobre concorrência, processos e o desafio de manter a coerência entre a lógica e a interface em tempo real.

9. Conclusões

Este projeto representou uma oportunidade significativa para consolidar conhecimentos sobre programação concorrente e sincronização de processos em Python. Ao longo do desenvolvimento, foi possível compreender na prática os desafios envolvidos na coordenação entre múltiplos processos, especialmente quando se trabalha com filas de prioridade, estados compartilhados e comunicação entre diferentes partes do sistema. A escolha de simular um ambiente hospitalar permitiu criar um cenário realista, dinâmico e didático, no qual os conceitos aprendidos puderam ser aplicados de forma contextualizada.